Análise Comparativa de Algoritmos de Ordenação em Estruturas de Dados Dinâmicas e Estáticas

Elder Rayan Oliveira Silva, Samuel Wagner Tiburi Silveira, Diogo Gomes Figueiredo,
Manoel Junio Duarte da Silva, Pedro Yan Alcantara Palácio, Antonio Pereira da Luz Neto
Universidade Federal do Cariri – UFCA
Engenharia de Software
Disciplina: Estrutura de Dados

***Resumo*—Este artigo compara o desempenho de cinco algoritmos de ordenação (Bubble, Selection, Insertion, Quick e Merge Sort) em três estruturas de dados: vetor, lista duplamente encadeada dinâmica e lista estática indexada. O contexto é um sistema de ranking de alunos por nota. Os experimentos foram escritos em C e executados 100 vezes por cenário, variando o tamanho da entrada ($N = 100$, 1.000 e 10.000), o padrão dos dados (aleatório, ordenado e invertido) e a estrutura utilizada. Os resultados mostram vantagem consistente dos algoritmos $O(n \log n)$ para volumes maiores: com $N = 10.000$ e dados aleatórios no vetor, Quick Sort e Merge Sort ficaram próximos de 1 ms, enquanto os algoritmos quadráticos ultrapassaram dezenas de milissegundos. A estrutura também fez diferença — o vetor foi o mais rápido, a lista dinâmica ficou no meio-termo e a lista estática foi a mais lenta na maioria dos cenários. O Merge Sort foi escolhido como a alternativa mais robusta por manter desempenho previsível em qualquer tipo de entrada.**

***Index Terms*—Algoritmos de ordenação, estruturas de dados, lista encadeada, análise de desempenho, complexidade computacional.**

I. Introdução

A escolha conjunta de algoritmo de ordenação e estrutura de dados afeta diretamente o tempo de resposta de sistemas reais [1], [2]. Em aplicações com atualização frequente de rankings, esse custo passa a ser um fator crítico.

A análise assintótica orienta a comparação teórica, mas fatores como localidade de cache e custo de navegação na estrutura alteram o desempenho observado na prática [3], [4]. Por isso, este trabalho combina fundamentação teórica e medição experimental.

O objetivo é comparar cinco algoritmos clássicos (Bubble, Selection, Insertion, Quick e Merge Sort) em três estruturas (vetor, lista dinâmica e lista estática), considerando diferentes tamanhos e padrões de entrada, e identificar a combinação mais adequada para um sistema de ranking de alunos.

II. Fundamentação Teórica

A. Estruturas Avaliadas

Foram implementadas três estruturas para armazenar registros de alunos (id e nota):

1) Vetor Convencional: Memória contígua com acesso direto por índice em $O(1)$ e alta localidade de cache.

2) Lista Duplamente Encadeada Dinâmica: Nós alocados sob demanda com `malloc()`, flexível em tamanho, porém com pior localidade de cache devido aos nós espalhados na memória.

3) Lista Duplamente Encadeada Estática: Simula encadeamento em vetor pré-alocado usando índices inteiros. Evita alocação dinâmica por nó, mas adiciona custo de navegação indireta.

B. Algoritmos de Ordenação

Foram avaliados Bubble, Selection, Insertion, Quick e Merge Sort. Os três primeiros são quadráticos no caso médio/pior; Quick e Merge são $O(n \log n)$ em média, com Merge mantendo esse limite também no pior caso [1], [3]. A Tabela I resume as complexidades.

Tabela I
Complexidade de tempo e espaço dos algoritmos

Algoritmo	Melhor	Médio	Pior	Espaço
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

III. Metodologia

O experimento simula um sistema de ranking de alunos ordenados por nota, implementado em C com análise dos resultados em Python. As variáveis controladas foram:

- **Volume (N):** 100 (pequeno), 1.000 (médio) e 10.000 (grande) elementos.
- **Tipo de entrada:** aleatório, ordenado e inversamente ordenado.
- **Estrutura:** vetor convencional, lista dinâmica e lista estática.

Cada combinação foi executada 100 vezes. O tempo medido inclui apenas a ordenação, com a média em milissegundos como métrica principal.

IV. Implementação

Cada algoritmo foi implementado em versões específicas para cada estrutura, mantendo o mesmo critério de ordenação (nota crescente) e o mesmo tipo de registro `Aluno`. No vetor, as trocas usam acesso direto por índice; na lista dinâmica, a navegação ocorre por ponteiros; na lista estática, por índices que simulam ponteiros. A semântica de estabilidade foi mantida quando aplicável, especialmente no Merge Sort.

A medição foi feita apenas no trecho de ordenação, excluindo geração de entrada e operações de E/S, para que os valores reflitam o custo algorítmico e o custo de acesso estrutural.

V. Resultados e Análise

A. Comparação Geral de Desempenho

Com $N = 10.000$ e entrada aleatória no vetor, Quick Sort (1,01 ms) e Merge Sort (1,36 ms) ficaram muito abaixo dos algoritmos quadráticos: Insertion (40,14 ms), Selection (54,85 ms) e Bubble (142,05 ms). Esse resultado confirma, na prática, a vantagem assintótica dos algoritmos $O(n \log n)$ para volumes maiores.

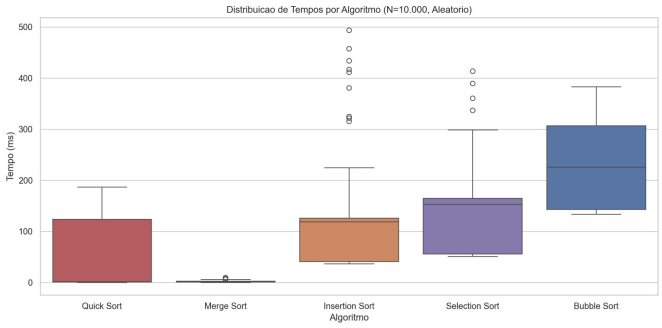

Figura 1. Comparação geral de desempenho ($N = 10.000$, entrada aleatória no vetor).

B. Tipo de Entrada e Estrutura de Dados

O Quick Sort, com pivô no último elemento, teve ótimo desempenho no caso aleatório, mas degradou fortemente em entradas ordenadas e invertidas. O Merge Sort se mantém estável nos três cenários. Quanto às estruturas, o vetor foi o mais rápido de forma consistente, a lista dinâmica apresentou overhead moderado e a lista estática obteve os maiores tempos na maioria dos cenários.

C. Curva de Crescimento

Para $N = 100$ e $N = 1.000$, as diferenças são pequenas. Em $N = 10.000$, a separação entre classes de complexidade fica evidente: os métodos quadráticos crescem rapidamente, enquanto Merge e Quick permanecem em faixa muito inferior.

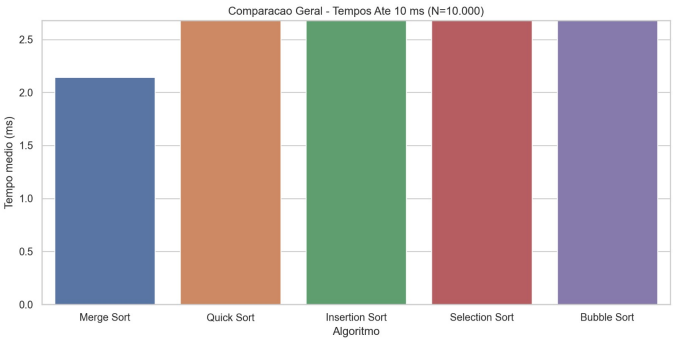

Figura 2. Desempenho por tipo de entrada para $N = 10.000$.

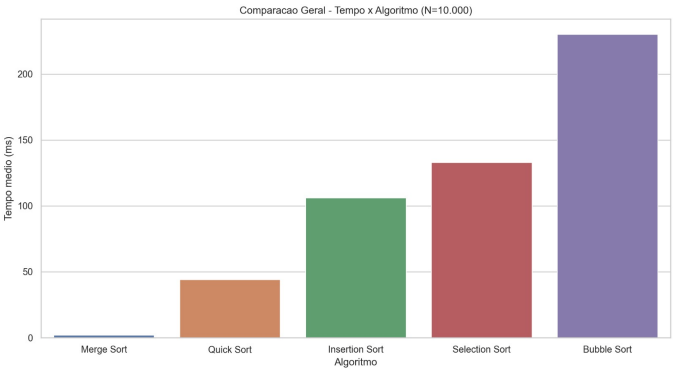

Figura 3. Curva de crescimento do tempo médio em função de N.

D. Comparativo Entre Estruturas

Mesmo com o mesmo algoritmo, a diferença entre as estruturas pode ser expressiva. O Quick Sort no vetor levou 1,01 ms; na lista dinâmica, 128,97 ms, pois a navegação por ponteiros elimina a vantagem do acesso aleatório. Esse resultado reforça que a escolha da estrutura não é detalhe de implementação, mas parte central da decisão de projeto.

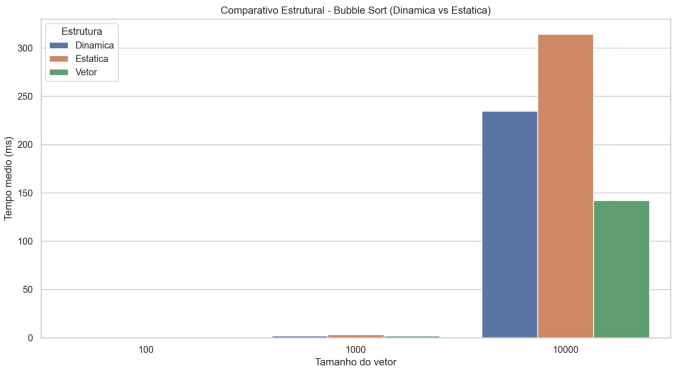

Figura 4. Comparação por estrutura de dados ($N = 10.000$, entrada aleatória).

E. Variabilidade dos Tempos

O Merge Sort mantém baixa variabilidade entre execuções, enquanto o Quick Sort apresenta maior espalhamento nos

cenários desfavoráveis. Para aplicações reais, essa previsibilidade importa tanto quanto a média.

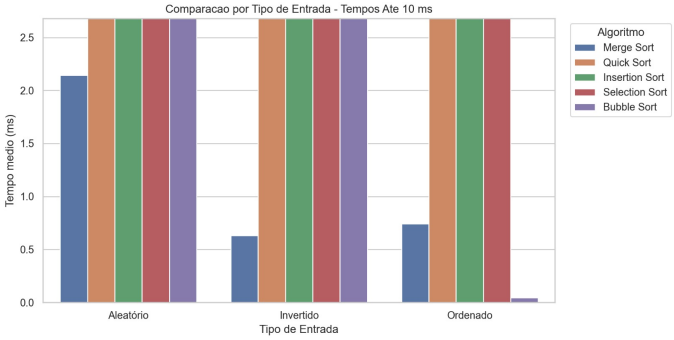

Figura 5. Distribuição dos tempos por algoritmo para $N = 10.000$.

F. Síntese Numérica

Tabela II
TEMPO MÉDIO (MS) PARA $N = 10.000$, ENTRADA ALEATÓRIA

Algoritmo	**Vetor**	**Dinâmica**	**Estática**
Bubble Sort	142,05	234,55	314,27
Selection Sort	54,85	158,12	186,32
Insertion Sort	40,14	125,79	152,50
Quick Sort	1,01	128,97	2,49
Merge Sort	1,36	1,28	3,79

VI. DISCUSSÃO

Os resultados reforçam três pontos. Primeiro, o ganho dos algoritmos $O(n \log n)$ só aparece com clareza em volumes maiores, para $N = 100$, qualquer algoritmo resolve bem. Segundo, a estrutura de dados altera significativamente o tempo final, mesmo para o mesmo algoritmo. Terceiro, o Quick Sort depende da estratégia de pivô: sem mitigação, degradou em padrões desfavoráveis de entrada.

Do ponto de vista de aplicação, o comportamento mais estável vale mais do que o menor tempo em um único cenário. Embora o Quick Sort tenha sido excelente no caso aleatório em vetor, o Merge Sort se mostrou mais confiável ao manter desempenho consistente em todos os perfis de entrada. Estruturas com navegação indireta penalizam principalmente os algoritmos quadráticos, que ampliam o custo adicional de acesso à medida que N cresce.

O Quick Sort foi configurado com pivô fixo no último elemento, o que acentua o pior caso para entradas ordenadas. Estratégias como pivô aleatório ou mediana-de-três produziriam comportamento mais equilibrado. A métrica baseada em `clock()` pode ter resolução insuficiente para entradas pequenas, gerando valores próximos de zero. Além disso, os experimentos foram conduzidos em um único ambiente computacional, o que pode influenciar os resultados absolutos. Ainda assim, a comparação relativa entre algoritmos e estruturas se manteve consistente ao longo dos cenários.

VII. CONCLUSÃO

O estudo confirma que algoritmo e estrutura devem ser escolhidos em conjunto. O Merge Sort apresentou o melhor equilíbrio entre desempenho, estabilidade e previsibilidade, sendo a opção mais segura para o problema de ranking de alunos. O Quick Sort foi competitivo no caso médio, mas sensível ao padrão de entrada com pivô fixo.

Como continuidade, recomenda-se testar estratégias de pivô para o Quick Sort, ampliar os tamanhos de entrada e incluir métricas complementares (comparações, trocas e uso de memória) para refinar a análise.

REFERÊNCIAS

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Algoritmos: Teoria e Prática*, 3ª ed. Rio de Janeiro: Elsevier, 2012.

[2] R. Sedgewick e K. Wayne, *Algorithms*, 4ª ed. Boston: Addison-Wesley, 2011.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2ª ed. Boston: Addison-Wesley, 1998.

[4] N. Ziviani, *Projeto de Algoritmos com Implementações em C e Pascal*, 3ª ed. São Paulo: Cengage Learning, 2010.